

## 家庭作业

- 4.45 在 3.4.2 节中, x86-64 `pushq` 指令被描述成要减少栈指针, 然后将寄存器存储在栈指针的位置。因此, 如果我们有一条指令形如对于某个寄存器 `REG`, `pushq REG`, 它等价于下面的代码序列:

```
subq $8,%rsp      Decrement stack pointer
movq REG, (%rsp)  Store REG on stack
```

- A. 借助于练习题 4.7 中所做的分析, 这段代码序列正确地描述了指令 `pushq %rsp` 的行为吗? 请解释。
- B. 你该如何改写这段代码序列, 使得它能够像对 `REG` 是其他寄存器时一样, 正确地描述 `REG` 是 `%rsp` 的情况?

- 4.46 在 3.4.2 节中, x86-64 `popq` 指令被描述为将来自栈顶的结果复制到目的寄存器, 然后将栈指针减少。因此, 如果我们有一条指令形如 `popq REG`, 它等价于下面的代码序列:

```
movq (%rsp), REG  Read REG from stack
addq $8,%rsp      Increment stack pointer
```

- A. 借助于练习题 4.8 中所做的分析, 这段代码序列正确地描述了指令 `popq %rsp` 的行为吗? 请解释。
- B. 你该如何改写这段代码序列, 使得它能够像对 `REG` 是其他寄存器时一样, 正确地描述 `REG` 是 `%rsp` 的情况?

- 4.47 你的作业是写一个执行冒泡排序的 Y86-64 程序。下面这个 C 函数用数组引用实现冒泡排序, 供你参考:

```
1  /* Bubble sort: Array version */
2  void bubble_a(long *data, long count) {
3      long i, last;
4      for (last = count-1; last > 0; last--) {
5          for (i = 0; i < last; i++)
6              if (data[i+1] < data[i]) {
7                  /* Swap adjacent elements */
8                  long t = data[i+1];
9                  data[i+1] = data[i];
10                 data[i] = t;
11             }
12     }
13 }
```

- A. 书写并测试一个 C 版本, 它用指针引用数组元素, 而不是用数组索引。
- B. 书写并测试一个由这个函数和测试代码组成的 Y86-64 程序。你会发现模仿编译你的 C 代码产生的 x86-64 代码来做实现会很有帮助。虽然指针比较通常是用无符号算术运算来实现的, 但是在这个练习中, 你可以使用有符号算术运算。

- 4.48 修改对家庭作业 4.47 所写的代码, 实现冒泡排序函数的测试和交换(6~11 行), 要求不使用跳转, 且最多使用 3 次条件传送。

- 4.49 修改对家庭作业 4.47 所写的代码, 实现冒泡排序函数的测试和交换(6~11 行), 要求不使用跳转, 且只使用 1 次条件传送。

- 4.50 在 3.6.8 节中, 我们看到实现 `switch` 的一种常见方法是创建一组代码块, 再用跳转表对这些块进行索引。考虑图 4-69 中给出的函数 `switchv` 的 C 代码, 以及相应的测试代码。

用跳转表以 Y86-64 实现 `switchv`。虽然 Y86-64 指令集不包含间接跳转指令, 但是, 你可以通过把计算好的地址入栈, 再执行 `ret` 指令来获得同样的效果。实现类似于 C 语言所示的测试代码, 证明你的 `switchv` 实现可以处理触发 `default` 的情况以及两个显式处理的情况。

```

#include <stdio.h>
/* Example use of switch statement */

long switchv(long idx) {
    long result = 0;
    switch(idx) {
        case 0:
            result = 0xaaa;
            break;
        case 2:
        case 5:
            result = 0xbbb;
            break;
        case 3:
            result = 0xccc;
            break;
        default:
            result = 0xddd;
    }
    return result;
}

/* Testing Code */
#define CNT 8
#define MINVAL -1

int main() {
    long vals[CNT];
    long i;
    for (i = 0; i < CNT; i++) {
        vals[i] = switchv(i + MINVAL);
        printf("idx = %ld, val = 0x%lx\n", i + MINVAL, vals[i]);
    }
    return 0;
}

```

图 4-69 Switch 语句可以翻译成 Y86-64 代码。这要求实现一个跳转表

- \* 4.51 练习题 4.3 介绍了 `iaddq` 指令，即将立即数与寄存器相加。描述实现该指令所执行的计算。参考 `irmovq` 和 `OPq` 指令的计算(图 4-18)。
- \*\* 4.52 文件 `seq-full.hcl` 包含 `SEQ` 的 HCL 描述，并将常数 `IIADDQ` 声明为十六进制值 `C`，也就是 `iaddq` 的指令代码。修改实现 `iaddq` 指令的控制逻辑块的 HCL 描述，就像练习题 4.3 和家庭作业 4.51 中描述的那样。可以参考实验资料获得如何为你的解答生成模拟器以及如何测试模拟器的指导。
- \*\* 4.53 假设要创建一个较低成本的、基于我们为 PIPE 设计的结构(图 4-41)的流水线化的处理器，不使用旁路技术。这个设计用暂停来处理所有的数据相关，直到产生所需值的指令已经通过了写回阶段。

文件 `pipe-stall.hcl` 包含一个对 PIPE 的 HCL 代码的修改版，其中禁止了旁路逻辑。也就是，信号 `e_valA` 和 `e_valB` 只是简单地声明如下：

```

## DO NOT MODIFY THE FOLLOWING CODE.
## No forwarding. valA is either valP or value from register file
word d_valA = [
    D_icode in { ICALL, IJXX } : D_valP; # Use incremented PC
    1 : d_rvalA; # Use value read from register file
];

```



```
## No forwarding. valB is value from register file
word d_valB = d_rvalB;
```

修改文件结尾处的流水线控制逻辑，使之能正确处理所有可能的控制和数据冒险。作为设计工作的一部分，你应该分析各种控制情况的组合，就像我们在 PIPE 的流水线控制逻辑设计中做的那样。你会发现有许多不同的组合，因为有更多的情况需要流水线暂停。要确保你的控制逻辑能正确处理每种组合情况。可以参考实验资料指导你如何为解答生成模拟器以及如何测试模拟器的。

\*\*4.54 文件 pipe-full.hcl 包含一份 PIPE 的 HCL 描述，以及常数值 IIADDQ 的声明。修改该文件以实现指令 iaddq，就像练习题 4.3 和家庭作业 4.51 中描述的那样。可以参考实验资料获得如何为你的解答生成模拟器以及如何测试模拟器的指导。

\*\*4.55 文件 pipe-nt.hcl 包含一份 PIPE 的 HCL 描述，并将常数 J\_YES 声明为值 0，即无条件转移指令的功能码。修改分支预测逻辑，使之对条件转移预测为不选择分支，而对无条件转移和 call 预测为选择分支。你需要设计一种方法来得到跳转目标地址 valC，并送到流水线寄存器 M，以便从错误的分支预测中恢复。可以参考实验资料获得如何为你的解答生成模拟器以及如何测试模拟器的指导。

\*\*4.56 文件 pipe-btfmt.hcl 包含一份 PIPE 的 HCL 描述，并将常数 J\_YES 声明为值 0，即无条件转移指令的功能码。修改分支预测逻辑，使得当  $valC < valP$  时(后向分支)，就预测条件转移为选择分支，当  $valC \geq valP$  时(前向分支)，就预测为不选择分支。(由于 Y86-64 不支持无符号运算，你应该使用有符号比较来实现这个测试。)并且将无条件转移和 call 预测为选择分支。你需要设计一种方法来得到 valC 和 valP，并送到流水线寄存器 M，以便从错误的分支预测中恢复。可以参考实验资料获得如何为你的解答生成模拟器以及如何测试模拟器的指导。

\*\*4.57 在我们的 PIPE 的设计中，只要一条指令执行了 load 操作，从内存中读一个值到寄存器，并且下一条指令要用这个寄存器作为源操作数，就会产生一个暂停。如果要在执行阶段中使用这个源操作数，暂停是避免冒险的唯一方法。对于第二条指令将源操作数存储到内存的情况，例如 rmmovq 或 pushq 指令，是不需要这样的暂停的。考虑下面这段代码示例：

```
1    mrmovq 0(%rcx),%rdx    # Load 1
2    pushq %rdx             # Store 1
3    nop
4    popq %rdx              # Load 2
5    rmmovq %rax,0(%rdx)    # Store 2
```

在第 1 行和第 2 行，mrmovq 指令从内存读一个值到 %rdx，然后 pushq 指令将这个值压入栈中。我们的 PIPE 设计会让 pushq 指令暂停，以避免装载/使用冒险。不过，可以看到，pushq 指令要到访存阶段才会需要 %rdx 的值。我们可以再添加一条旁路通路，如图 4-70 所示，将内存输出(信号 m\_valM)转发到流水线寄存器 M 中的 valA 字段。在下一个时钟周期，被传送的值就能写入内存了。这种技术称为加载转发(load forwarding)。

注意，上述代码序列中的第二个例子(第 4 行和第 5 行)不能利用加载转发。popq 指令加载的值是作为下一条指令地址计算的一部分的，而在执行阶段而非访存阶段就需要这个值了。

A. 写出描述发现加载/使用冒险条件的逻辑公式，类似于图 4-64 所示，除了能用加载转发时不会导致暂停以外。

B. 文件 pipe-1f.hcl 包含一个 PIPE 控制逻辑的修改版。它含有信号 e\_valA 的定义，用来实现图 4-70 中标号为“Fwd A”的块。它还将流水线控制逻辑中的加载/使用冒险的条件设置为 0，因此流水线控制逻辑将不会发现任何形式的加载/使用冒险。修改这个 HCL 描述以实现加载转发。可以参考实验资料获得如何为你的解答生成模拟器以及如何测试模拟器的指导。

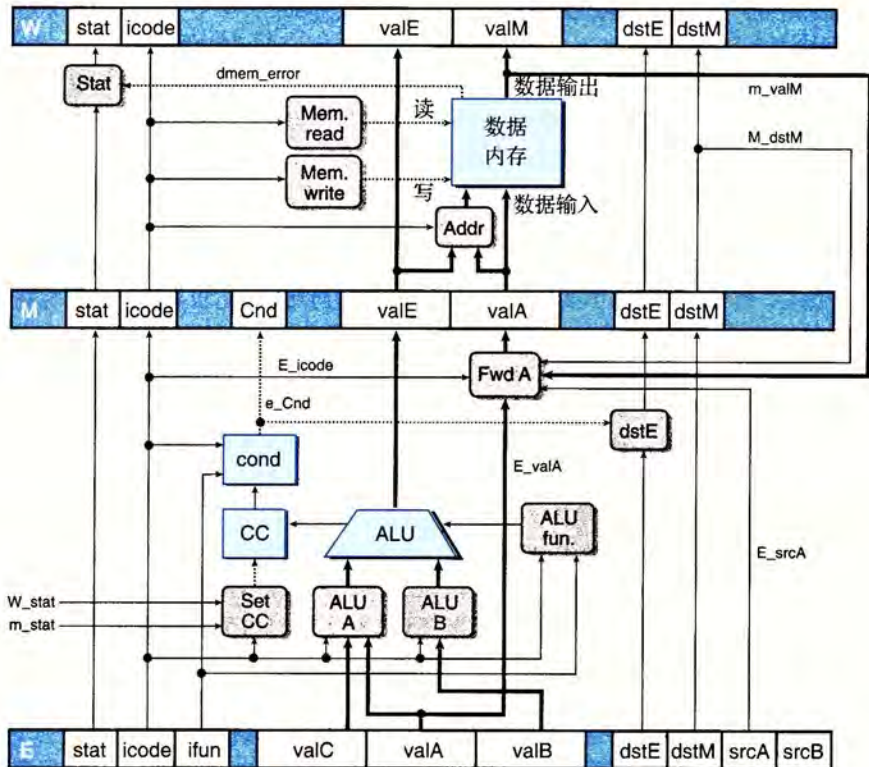
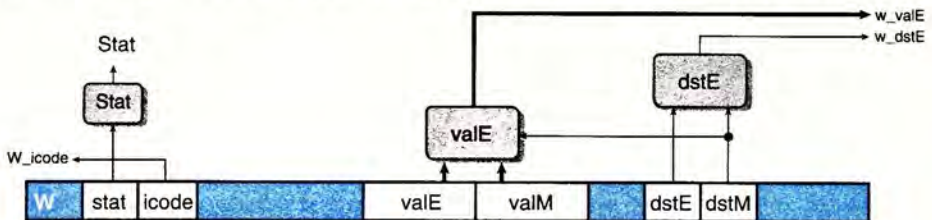


图 4-70 能够进行加载转发的执行和访存阶段。通过添加一条从内存输出到流水线寄存器 M 中 valA 的源的旁路通路，对于这种形式的加载/使用冒险，我们可以使用转发而不必暂停。这是家庭作业 4.57 的主旨

**\*\* 4.58** 我们的流水线化的设计有点不太现实，因为寄存器文件有两个写端口，然而只有 popq 指令需要对寄存器文件同时进行两个写操作。因此，其他指令只使用一个写端口，共享这个端口来写 valE 和 valM。下面这个图是一个对写回逻辑的修改版，其中，我们将写回寄存器 ID (W\_dstE 和 W\_dstM) 合并成一个信号 w\_dstE，同时也将写回值 (W\_valE 和 W\_valM) 合并成一个信号 w\_valE:



用 HCL 写执行这些合并的逻辑，如下所示；

```
## Set E port register ID
word w_dstE = [
    ## writing from valM
    W_dstM != RNONE : W_dstM;
    1: W_dstE;
];

## Set E port value
word w_valE = [
    W_dstM != RNONE : W_valM;
    1: W_valE;
];
```



对这些多路复用器的控制是由 `dstE` 确定的——当它表明有某个寄存器时，就选择端口 E 的值，否则就选择端口 M 的值。

在模拟模型中，我们可以禁止寄存器端口 M，如下面这段 HCL 代码所示：

```
## Disable register port M
## Set M port register ID
word w_dstM = RNONE;

## Set M port value
word w_valM = 0;
```

接下来的问题就是要设计处理 `popq` 的方法。一种方法是用控制逻辑动态地处理指令 `popq rA`，使之与下面两条指令序列有一样的效果：

```
iaddq $8, %rsp
mrmovq -8(%rsp), rA
```

(关于指令 `iaddq` 的描述，请参考练习题 4.3) 要注意两条指令的顺序，以保证 `popq %rsp` 能正常工作。要达到这个目的，可以让译码阶段的逻辑对上面列出的 `popq` 指令和 `addq` 指令一视同仁，除了它会预测下一个 PC 与当前 PC 相等以外。在下一个周期，再次取出了 `popq` 指令，但是指令代码变成了特殊的值 `IPOP2`。它会被当作一条特殊的指令来处理，行为与上面列出的 `mrmovq` 指令一样。

文件 `pipe-lw.hcl` 包含上面讲的修改过的写端口逻辑。它将常数 `IPOP2` 声明为十六进制值 E。还包括信号 `f_icode` 的定义，它产生流水线寄存器 D 的 `icode` 字段。可以修改这个定义，使得当第二次取出 `popq` 指令时，插入指令代码 `IPOP2`。这个 HCL 文件还包含信号 `f_pc` 的声明，也就是标号为“Select PC”的块(图 4-57)在取指阶段产生的程序计数器的值。

修改该文件中的控制逻辑，使之按照我们描述的方式来处理 `popq` 指令。可以参考实验资料获得如何为你的解答生成模拟器以及如何测试模拟器的指导。

- 4.59 比较三个版本的冒泡排序的性能(家庭作业 4.47、4.48 和 4.49)。解释为什么一个版本的性能比其他两个的好。

## 练习答案

- 4.1 手工对指令编码是非常乏味的，但是它将巩固你对汇编器将汇编代码变成字节序列的理解。在下面这段 Y86-64 汇编器的输出中，每一行都给出了一个地址和一个从该地址开始的字节序列：

```
1  0x100:                                | .pos 0x100 # Start code at address
0x100
2  0x100: 30f30f000000000000000000 |    irmovq $15,%rbx
3  0x10a: 2031                        |    rrmovq %rbx,%rcx
4  0x10c:                                | loop:
5  0x10c: 4013fdfffffffffffffff |    rmmovq %rcx,-3(%rbx)
6  0x116: 6031                        |    addq %rbx,%rcx
7  0x118: 700c010000000000000000 |    jmp loop
```

这段编码有些地方值得注意：

- 十进制的 15(第 2 行)的十六进制表示为 `0x000000000000000f`。以反向顺序来写就是 `0f 00 00 00 00 00 00 00`。
- 十进制 -3(第 5 行)的十六进制表示为 `0xfffffffffffffffd`。以反向顺序来写就 `fd ff ff ff ff ff ff ff`。
- 代码从地址 `0x100` 开始。第一条指令需要 10 个字节，而第二条需要 2 个字节。因此，循环的目标地址为 `0x0000010c`。以反向顺序来写就是 `0c 01 00 00 00 00 00 00`。

- 4.2 手工对一个字节序列进行译码能帮助你理解处理器面临的任務。它必须读入字节序列，并确定要执行什么指令。接下来，我们给出的是用来产生每个字节序列的汇编代码。在汇编代码的左边，你可以看到每条指令的地址和字节序列。